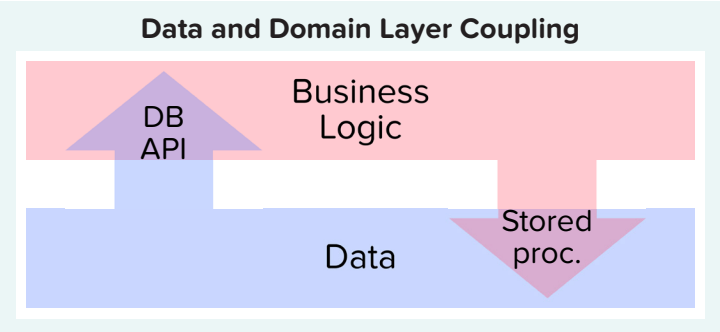
ENTERPRISE APPLICATION INFRASTRUCTURE

Working with Databases

Impedance mismatch: The set of inherent differences in the way data is represented between the business logic, which is often an object-oriented program, and the data store, which is an SQL-based relational database.

There are two basic ways to remove SQL from the business logic, but they both pose problems with maintainability, scalability, and high coupling.

1. **Stored procedures:** Pre-written SQL statements stored in the database that business logic can invoke with parameters.
2. **Database connectivity APIs:** Decouple the specific database platform from application code that accesses it.



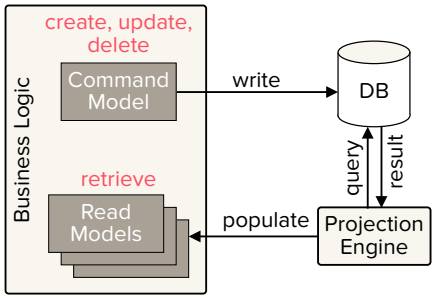
An **object-relational mapping (ORM)** tool represents the database via objects and handles translations entailed by impedance mismatch.

Note: ORM is a better solution to impedance mismatch than stored procedures or database connectivity APIs.

Command-Query Responsibility Segregation (CQRS): A pattern that splits operations that read data from those that write it.

In CQRS:

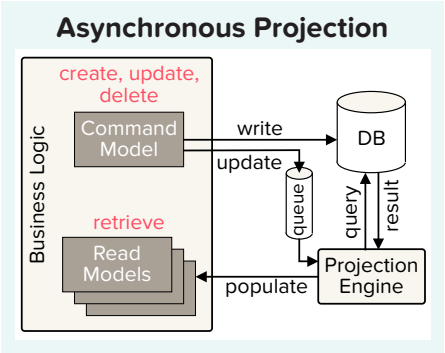
- All changes to the database are made by the command model in the business logic.
- All readable data is retrieved and stored in one or more read models, which is populated by a projection engine.



©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

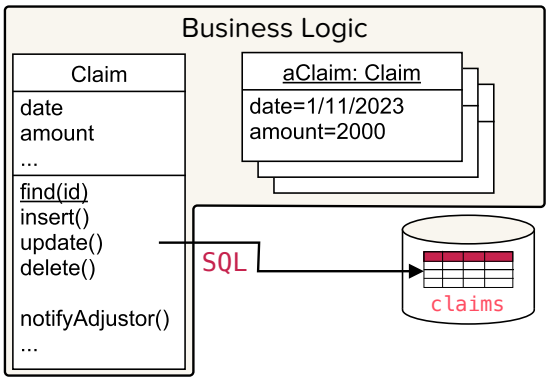
There are two models for consistency in a distributed data store (i.e., the database and projection engine).

- To get strong consistency, data should be kept consistent across multiple elements. CQRS achieves strong consistency via **synchronous projection**, a timestamp-based solution.
- Eventual consistency: An approach in which different elements in a distributed data store are eventually updated with new data. In CQRS, eventual consistency is implemented via **asynchronous projection**, which sends updates directly to the projection engine in addition to writing to the database.



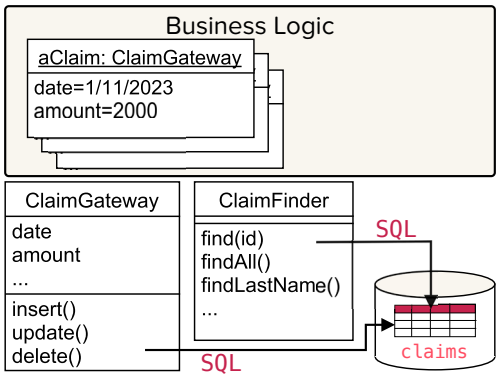
Data-Source Patterns

Active Record: The simplest data-source pattern in which rows from the database are represented as domain objects in business logic.



Row Data Gateway: A pattern in which the gateway is a class whose instances each represent a single row in a database table.

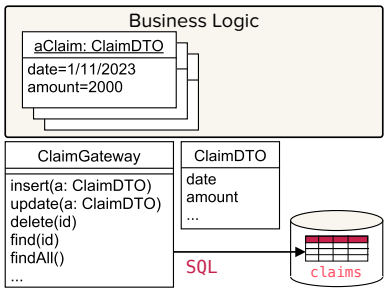
The Row Data Gateway pattern has methods to create, update, and delete information, along with a finder class that has methods to read data from the database.



©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Table Data Gateway: A pattern that uses a single instance of a gateway class to access the table.

- The gateway class has methods to both read and update the table.
- Table rows are represented by **data transfer objects (DTOs)**, which only have attributes.
- To update the database, business logic passes the entire DTO to the gateway instance.



Data-Source & Business-Logic Pattern Pairings

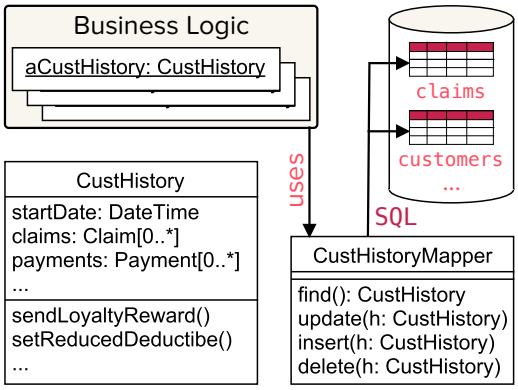
Data-Source Pattern	Business-Logic Pattern(s)	Details
Active Record	Domain Model or Transaction Script	These patterns work well together if the domain objects are closely aligned to the database structure and their relationships aren't too complex.
Row Data Gateway	Transaction Script	These patterns work well together when the business logic involves relatively few transactions that are less complex.
Table Data Gateway	Transaction Script or Table Module	These patterns work well together because, in Transaction Script, procedures invoke methods in the gateway instance and work with the DTOs they provide. In Table Module, the business logic corresponds closely with the database table layout.

©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Advanced Data-Source Patterns

Data Mapper: A pattern that removes the business logic's dependency on the database schema using a layer of software called a mapper class.

The mapper class lies between the business logic and the database.

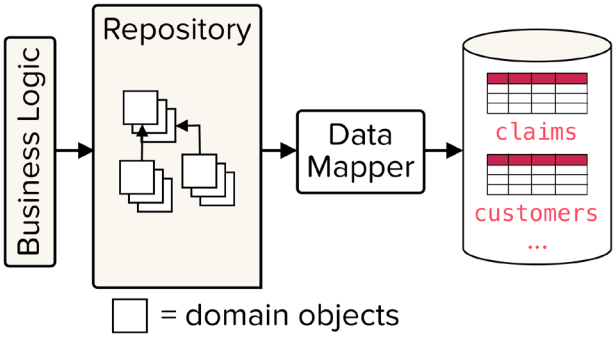


Important: Data Mapper replaces the responsibilities of Row and Table Data Gateway for database access and object instantiation. In addition, code in the mapper translates between the structure of the business logic and the schema of the database.

Persistence ignorance: The ability to completely isolate the business logic from database persistence.

Repository: A pattern that helps achieve persistence ignorance by making persistent data appear to business logic like an in-memory collection of objects and completely abstracting away the data layer..

Complex queries are executed by the repository instead of in business logic or in a data mapper.



Note: You should consider complex patterns like Data Mapper and Repository only when the complexity of your business logic calls for them.

©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Deconfliction and Synchronization

Identity Map: A pattern that keeps a record of the domain objects that have been read from the database for a given transaction.

By keeping each and every object loaded in a map, the identity map pattern ensures that no object is loaded more than once.

Object graphs: Groups of objects that are connected to each other.

Eager loading: Querying a database for all the data needed to build the object graph.

Lazy loading: A strategy where only a small amount of memory is used at any given time because only domain objects that are actually needed are loaded into memory.

Unit of Work (UoW): A pattern that tracks all changes that occur in the business logic.

When a business logic transaction is successfully completed, UoW builds a database transaction to persist the results.

UoW controller: Compares the state of objects before and after a business transaction, and then issues all the change registration calls to the appropriate UoW objects.

Event-Driven Architecture

Messages: Data objects with information from a sender.

Event: A message that describes something that happened in the domain.

Events are posted to a **message bus**, which is responsible for calling one or more **handlers** that take appropriate action when particular events happen.

Event-Driven Architecture (EDA): A pattern that involves modules posting messages for receivers to act on, as opposed to calls between modules.

To maintain aggregate state in a full event-driven architecture, aggregates raise domain events for all their state changes and then persist them to an **event store**.

©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Consistency boundary: A contained set of logic that must either entirely succeed or completely fail.

Since events are meant to be handled outside the domain, they're outside the consistency boundary of a transaction and should be handled after the transaction is committed.

Command: A message requesting that the system perform a task and provide feedback on its success.

The feedback that goes back to the aggregate that issued the command should be handled as a separate transaction.

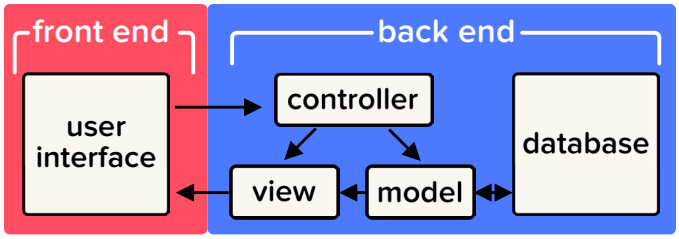
Event Sourcing pattern: Recreates the state of the aggregates at any point in their life by replaying the events that changed their state (in order, from first to last).

Presentation Patterns

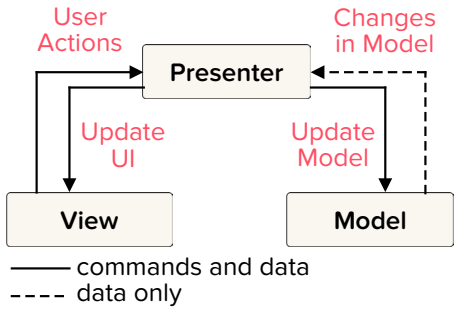
Model-View-Controller (MVC): A pattern that splits responsibilities for the user interface into three distinct roles.

There are two types of MVC controllers.

- A **page controller** is an object that handles requests for a specific page.
- A **front controller** handles requests for all the pages in an application using a single object.

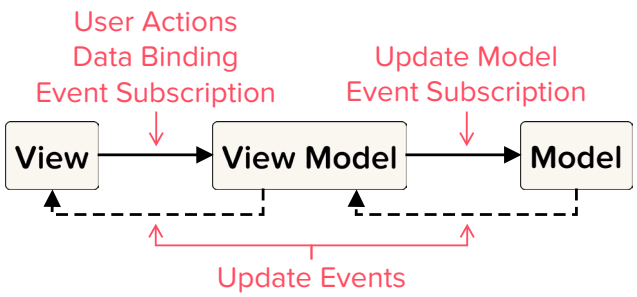


Model-View-Presenter (MVP): A pattern that renames the controller to the presenter and makes it an intermediary between the view and the model, shifting the view's dependency away from the model.



©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY

Model-View-ViewModel: A pattern that eliminates two-way dependencies and uses binding and events to enable automatic updates to the view.

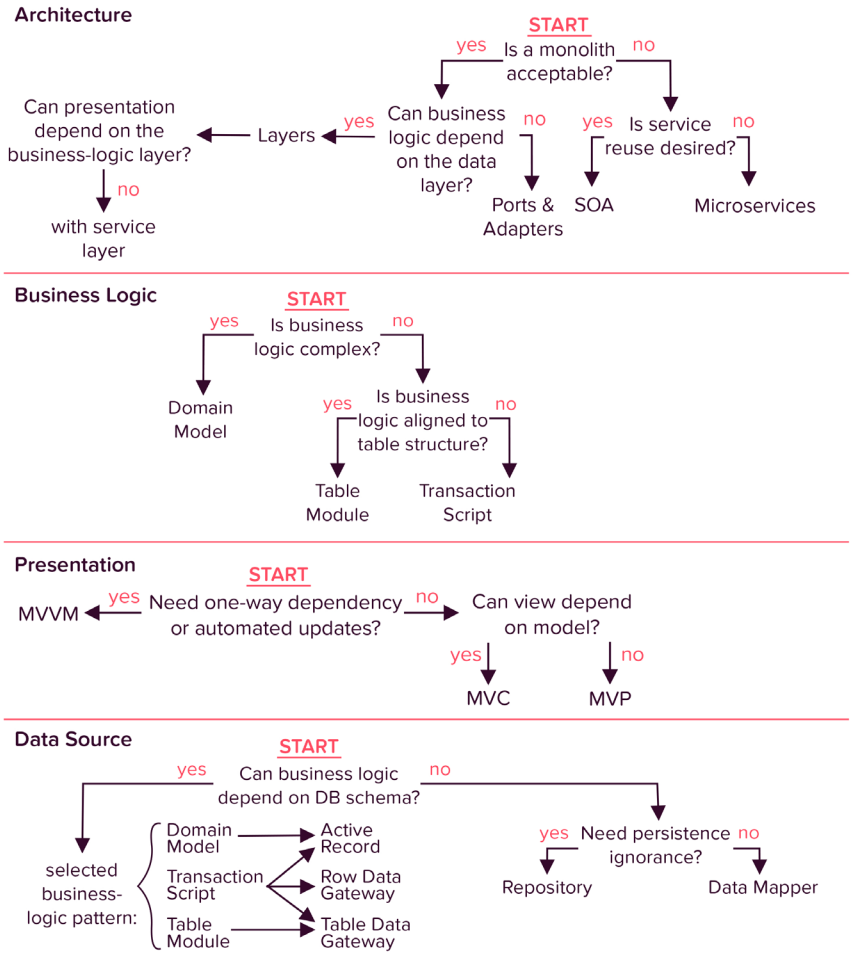


Single-Page Application (SPA): A model that loads just one web document for a given page and then updates using Ajax.

Remember: As you shift more responsibility for generating HTML from the server to the client, you move to SPA.

Choosing and Combining Patterns

Heuristic: A framework for making decisions that, while not always perfect, can help you identify solutions to problems.



©2023 QUANTIC SCHOOL OF BUSINESS AND TECHNOLOGY